ZELTA

Technical Documentation

A Financial Twin that tells you, in days, how long your money lasts, and checks a spend before you make it.

Version	Day 15 build (main, October 2026)
Repository	github.com/Tioluwanim/ZELTA (branches: main, frontend)
Status	Working prototype. Data is self-reported; no live bank connection yet.

1. Problem and audience

Many people in Nigeria live on money that has to stretch between irregular inflows (allowances, gigs, transfers) and fixed outflows (fees, rent, data, transport). Existing apps show balances and categories after the fact. They rarely answer the question that matters in the moment: **how many days does this money last, and is it safe to spend this amount now?**

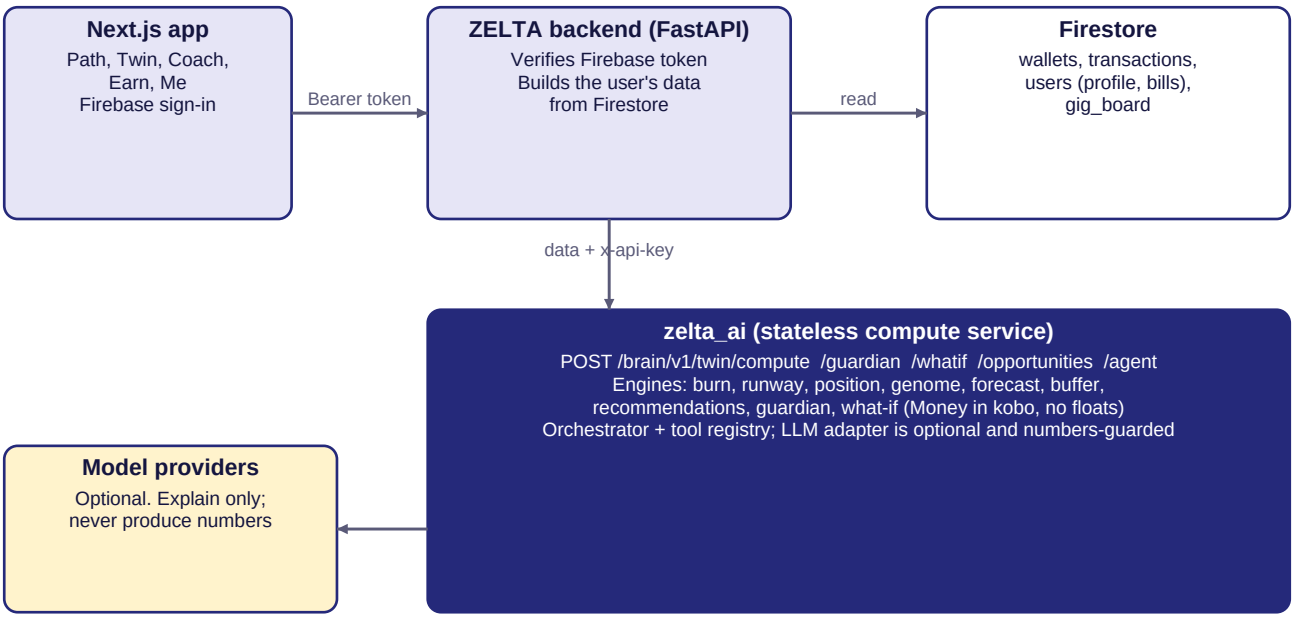
ZELTA builds a **Financial Twin** from what a person tells it (balance, income, spending, upcoming bills) and answers in plain days, with the reasoning shown. It started with students and is being generalised to anyone managing tight, irregular money.

Design principles

- **Math first, AI explains.** Every number comes from deterministic engines using integer kobo. A language model may only phrase a result, and a number guard rejects any figure it invents.
- **Unknown stays unknown.** Missing data is returned as null with a reason, never as zero.
- **No fabrication.** The service never invents opportunities, returns or bank connectivity. Data is labelled self_reported.
- **The user stays in control.** ZELTA never moves money.

2. Architecture

Three deployable parts and one datastore. The browser never talks to the compute service. The backend authenticates the user, assembles their data and calls the stateless AI service with an internal key; the service computes and stores nothing.



Layer	Technology	Responsibility
Frontend	Next.js 16, React 19, Tailwind 4, TypeScript, Firebase Auth	Screens, typed API client, demo mode, accessibility (WCAG-minded: focus, reduced motion, no colour-only meaning)
Backend	FastAPI (Python), Firestore, httpx	Auth, wallet and profile, builds the twin payload, strict request models, safe error messages
Compute service	FastAPI, pydantic, pure Python engines	Financial Twin engines, Guardian, what-if, opportunity ranking, tool-using agent
Model layer (optional)	Provider-agnostic adapter	Retry, circuit breaker, fallback, number guard, shadow mode to retire third-party models
Data	Firestore (backend only)	Wallets, transactions, profiles, gig board

3. How the codebase is structured for maintainability

Each concern lives in one place and depends only inward. Engines are pure functions of an explicit snapshot and an injected clock, so they are deterministic and cheap to test.

```
zelta_ai/
  domain/     Money (integer kobo), Lagos clock, events
  ledger/     dedupe, pending-to-posted, reversals, own transfers
  engines/    burn, runway, position, genome, forecast, buffer,
              recommendations, guardian, whatif
  financial_twin/  TwinSnapshot assembled from a provider
  providers/   FinancialProvider contract, mock, inline, Mono scaffold
  openbanking/  consent states, audit log, redaction, webhook auth, link flow
  agent_tools/  typed tool registry with permission scopes
  agents/     router, planner, conversation, orchestrator
  llm/        provider adapter, resilience, guard, shadow
  opportunities/  provenance-checked catalogue ranking
  api/         HTTP contract (compute.py, twin.py)
zelta_backend/  routes/ services/ schemas/ core/ middleware/
app/ components/ lib/  (Next.js frontend at the repo root)
scripts/local_e2e/  real-HTTP local integration harness
```

Practices that keep it maintainable

- **Pure, injected-clock engines** with explicit `as_of` and Africa/Lagos time, so results are reproducible.
- **Strict contracts:** pydantic models with `extra=forbid`, naira as decimal strings, bounded lists, floats rejected; one error shape {code, message}.
- **Typed tool registry:** the agent can only call registered tools with validated arguments and scopes (twin:read, twin:simulate, opportunities:read).
- **Provider boundary:** Mono, mock and self-reported data sit behind one FinancialProvider contract; a provider can be swapped without touching engines.
- **Tests close to behaviour:** 215 AI-service tests and 15 backend tests (engines, ledger rules, contract and security tests); the frontend is type-checked, linted and built in CI-style runs.
- **Docs next to code:** AUDIT, SECURITY, PENTEST, REGULATORY_MATRIX, LLM_ADAPTER and FRONTEND in zelta_ai/docs.

4. The engines

Engine	What it computes
burn / runway	Blended 28- and 7-day spend, day-by-day forecast with dated bills; runway in days with a stated confidence
position	Money in and out over 28 days, bills due, availability after bills, goal needs, safe flexible spending, 28-day daily spend
genome (habits)	Income timing, post-income spike, category share, weekend and month-end effect, recurring payments; thin data returns reasons, not guesses
forecast / buffer	Expected balance at 7, 14 and 30 days, likely run-out date and range, minimum safe amount and cushion
recommendations	Deterministic rules (prepare for a bill, spend less, set aside, build cushion) each with a because and withheld reasons
guardian	PROCEED, CAUTION or PROTECT for an action; the worst factor wins; unknown evidence is CAUTION, never PROCEED
what-if	Up to 5 bounded scenarios (spend less or more, receive, purchase, unexpected cost, save weekly) re-run through the same forecast
opportunities	Ranks a caller-supplied catalogue by runway pressure, source and risk; sourced items need a source, estimates need confidence

All thresholds and multipliers are **proposed**, not validated against real users.

5. API surface

Route	Purpose
GET /api/twin	Summary, runway, position, buffer, forecast, habits, recommendations for the signed-in user
POST /api/twin/guardian	Check one amount before spending
POST /api/twin/whatif	Run a bounded scenario
POST /api/twin/agent	One Coach turn through the orchestrator (suggestions, tools used)
GET /api/twin/opportunities	Ranked options from the open gig board, marked unverified
POST /api/wallet/intercept-check	Spend check backed by the Guardian (UNVERIFIED if the engine is unreachable)
/api/wallet, /api/profile, /api/copilot, /api/gig, /api/simulation	Wallet, profile, legacy copilot, gig board and savings projection used by the app
/brain/v1/* (compute service)	twin/compute, guardian, whatif, opportunities, agent: internal only, x-api-key and x-user-id

6. Security and privacy

- Compute service is secure by default: internal key required, constant-time compare, rate limit, dev routes closed unless APP_ENV=local.
- Stateless compute: nothing is stored there; the backend owns data.
- 500 responses never echo exception text (guarded by a test); user-facing messages are written, not raw.
- Bank-link consent lifecycle with least-privilege scopes, redacted audit entries, fail-closed webhook auth and idempotency.

- Secrets are wrapped so they cannot leak through logging; model output is checked so it cannot add numbers.

7. Running and testing

```
python -m pytest zelta_ai/tests/twin zelta_ai/tests/test_llm_adapter.py # compute service
python -m pytest zelta_backend/tests # backend
pnpm install && pnpm build # frontend (set NEXT_API_URL and NEXT_FIREBASE_*; NEXT_DEMO=1 for
sample data)
scripts/local_e2e/ # real backend + AI service on localhost for end-to-end checks
```

8. Limitations (stated plainly)

- No live bank data has flowed through the system. The Mono adapter is a scaffold; its field mapping and event names are unverified and no code exchange is wired.
- Balances and transactions are self-reported, so every output is labelled that way.
- Thresholds, blend weights and multipliers are proposals tested on synthetic data.
- Firebase sign-in and Firestore were not exercised in automated end-to-end runs; those steps were faked locally.
- Consent and audit stores are in memory; the backend data model is still partly student-specific.
- Not legal or financial advice, and not claimed to meet any regulatory standard; see docs/REGULATORY_MATRIX.md.

9. Roadmap

- Persist consent and audit in Firestore; verify Mono against a sandbox and wire the code exchange.
- Feed income streams and goals into the twin payload; persist Coach conversations.
- Validate thresholds with real, consented data; add a weekly review.